

Frontend Development Course

100-Day Day-Wise Lecture Plan | 200 Hours | 2 Hours/Day

B.Tech 3rd Year | Expanded from the 45-Day / 90-Hour Master Outline

This plan takes the original 45-day / 90-hour outline and stretches it to 100 teaching days (200 hours) by slowing the pace, adding dedicated practice/lab days, revision days, and a few extra industry-relevant modules —

TypeScript basics, Testing (Jest/RTL), and Next.js awareness — that give students broader exposure without changing the core HTML → CSS → JS → React → Redux → Project progression.

Module	Topic	Days	Hours
1	Web & Frontend Fundamentals	1-2	4
2	HTML5	3-7	10
3	CSS3	8-17	20
4	Responsive Design + CSS Frameworks (+ Sass)	18-22	10
5	JavaScript	23-38	32
6	DOM & Browser APIs	39-43	10
7	Modern JS, Async & REST APIs	44-49	12
8	Git & GitHub	50-51	4
9	TypeScript Basics (NEW)	52-54	6
10	React Fundamentals	55-68	28
11	Advanced React	69-76	16
12	Redux Toolkit	77-81	10
13	Testing — Jest & RTL (NEW)	82-84	6
14	Next.js Awareness (NEW)	85-86	4
15	Capstone Project + Deployment	87-96	20
16	Revision, Mock Interview & Presentation	97-100	8
	TOTAL	100 Days	200 Hours

Teaching ratio for every 2-hour class: ~45-60 min concept explanation, ~45-60 min hands-on coding, ~10-15 min questions/debugging. Students should spend at least 40% of total course time actually coding.

MODULE 1: Web & Frontend Fundamentals

Days 1-2 | 4 Hours | 2 Hours/Day

Day	Topics to be Covered	Practical / Output
1	What is Web Development? Internet vs Web. How websites work. Client vs Server. Frontend vs Backend.	
2	Browser architecture, HTTP/HTTPS basics, Request/Response cycle, URL/Domain/DNS, Static vs Dynamic websites, Frontend developer role, Frontend technology ecosystem (with diagram).	Set up dev environment: VS Code, browser DevTools, Node.js, Live Server extension

MODULE 2: HTML5

Days 3-7 | 10 Hours | 2 Hours/Day

Day	Topics to be Covered	Practical / Output
3	HTML introduction, document structure, elements & attributes, headings, paragraphs, lists, links.	
4	Images, comments, HTML entities. Tables: table, thead, tbody, tr, th, td, colspan, rowspan.	Personal Profile Page
5	HTML Forms: <form>, <input> types (text, email, password, number, radio, checkbox, date, file), select, textarea, button, labels, form attributes.	
6	Semantic HTML: header, nav, main, section, article, aside, footer, figure, figcaption. Accessibility basics: alt text, labels, semantic structure, keyboard accessibility.	Student Registration Form
7	HTML Project Day — review, debug and polish all HTML built so far.	Mini Project 1: College Student Portal (Home, About, Courses, Faculty, Students, Contact, Registration)

MODULE 3: CSS3

Days 8-17 | 20 Hours | 2 Hours/Day

Day	Topics to be Covered	Practical / Output
8	What is CSS? Inline / Internal / External CSS. Selectors: universal, element, class, ID, attribute.	
9	Colors, backgrounds, borders.	Style the Personal Profile Page using CSS
10	CSS Box Model: content, padding, border, margin, width, height, box-sizing (with diagram).	
11	Box model applied — building card-style layouts.	Product Card UI (start)
12	Typography: font-family, font-size, font-weight, line-height, text-align, text-decoration, overflow.	
13	display, position (static, relative, absolute, fixed, sticky), z-index.	Complete Product Card UI
14	Flexbox part 1: flex container, flex-direction, justify-content, align-items.	
15	Flexbox part 2: flex-wrap, gap, flex, align-self, order.	Build Navbar → Hero Section → 3/4 Product Cards → Footer using Flexbox
16	CSS Grid: grid container, grid-template-columns/rows, gap, grid-column, grid-row, repeat(), minmax(), auto-fit, auto-fill.	Dashboard Layout (Sidebar + Header + Content)
17	Advanced CSS: variables, pseudo-classes, pseudo-elements, transitions, transforms, animations, shadows, gradients, hover effects, specificity, inheritance, !important, debugging CSS with DevTools.	Mini Project 2: Modern Responsive Landing Page

MODULE 4: Responsive Web Design + CSS Frameworks

Days 18-22 | 10 Hours | 2 Hours/Day

Day	Topics to be Covered	Practical / Output
18	Responsive design principles, mobile-first approach, viewport meta tag.	
19	Media queries, breakpoints, responsive typography, responsive images, flexible layouts.	Begin converting Landing Page to responsive
20	Sass/SCSS basics: variables, nesting, partials, mixins (industry-relevant addition).	
21	CSS Framework Introduction — Bootstrap (grid system, components). Why frameworks are used + how to read/write them (not memorize).	
22	Tailwind CSS introduction: utility classes, config basics.	Convert Landing Page into a Fully Responsive Website (Bootstrap or Tailwind)

MODULE 5: JavaScript

Days 23-38 | 32 Hours | 2 Hours/Day

Day	Topics to be Covered	Practical / Output
23	JavaScript introduction, the JS engine, how JS runs in the browser, linking JS to HTML.	
24	Variables: var, let, const. Data types. Primitive vs Reference types.	
25	Type conversion, type coercion.	Practice exercises on data types & coercion
26	Operators: arithmetic, assignment, comparison, logical, ternary, nullish coalescing, optional chaining.	
27	Conditionals: if, else, else if, switch.	Student Grade Calculator
28	Loops: for, while, do-while, nested loops, break, continue.	
29	Arrays: creating arrays, accessing elements, updating elements.	
30	Array methods (part 1): push(), pop(), shift(), unshift(), slice(), splice(), includes(), indexOf().	
31	Functions: declaration, expression, arrow functions, parameters, arguments, return.	
32	Default parameters, rest parameters, spread operator, callback functions.	
33	Array methods (part 2 — very important for interviews & React): map(), filter(), reduce().	
34	Array methods (part 3): find(), findIndex(), some(), every(), sort(), forEach().	Exercises on a students[] dataset: filter, average, highest marks, sort, create new arrays
35	Objects: object creation, properties, methods, nested objects, destructuring.	
36	Object spread/rest, optional chaining. JSON: structure, JSON.parse(), JSON.stringify().	
37	Modern JS syntax: template literals, destructuring recap, modules (import/export).	

Day	Topics to be Covered	Practical / Output
38	Scope, hoisting, closures, 'this' keyword, higher-order functions.	Interview-concept revision + quiz: Scope, Closure, Hoisting, Execution Context, Call Stack, Callback, HOF

MODULE 6: DOM & Browser APIs

Days 39-43 | 10 Hours | 2 Hours/Day

Day	Topics to be Covered	Practical / Output
39	What is the DOM? DOM tree. Selecting elements: getElementById(), querySelector(), querySelectorAll().	
40	Changing content and styles, creating elements, removing elements, working with classes.	
41	Events: click, submit, input, change, mouse events, keyboard events, the event object.	
42	Event bubbling, event capturing, event delegation.	To-Do Application — Add Task, Delete Task
43	Browser storage: localStorage, sessionStorage, cookies basics.	Complete To-Do App (Complete Task, Edit Task, Filter Tasks) + save data in localStorage

MODULE 7: Modern JavaScript, Async & REST APIs

Days 44-49 | 12 Hours | 2 Hours/Day

Day	Topics to be Covered	Practical / Output
44	Form validation, custom validation, error messages — applied to the To-Do/registration forms.	
45	Asynchronous JavaScript: synchronous vs asynchronous, call stack, Web APIs, callback queue, event loop, setTimeout.	
46	Promises: promise states, .then(), .catch(), .finally().	
47	async / await, try/catch error handling.	
48	REST APIs: API, REST, HTTP methods (GET, POST, PUT, PATCH, DELETE), HTTP status codes, headers, request body, query parameters, JSON, fetch().	
49	Consuming a public API end-to-end.	Product Listing Application: fetch products, display cards, search, filter, delete UI item, loading state, error handling

MODULE 8: Git & GitHub

Days 50-51 | 4 Hours | 2 Hours/Day

Day	Topics to be Covered	Practical / Output
50	git init, git clone, git status, git add, git commit, git push, git pull, git branch, git checkout, git switch, git merge.	
51	GitHub repository, README, .gitignore, branching workflow, pull request basics.	Create GitHub portfolio repository; push all projects built so far

MODULE 9: TypeScript Basics (Industry Awareness)

Days 52-54 | 6 Hours | 2 Hours/Day

Day	Topics to be Covered	Practical / Output
52	Why TypeScript? Installing TS. Basic types: string, number, boolean, array, tuple, any.	
53	Interfaces, type aliases, typing functions, optional properties, readonly properties.	
54	TypeScript with React — typing props and state (awareness level, not full depth).	Convert one small existing JS file/component into TypeScript

MODULE 10: React Fundamentals

Days 55-68 | 28 Hours | 2 Hours/Day

Day	Topics to be Covered	Practical / Output
55	Why React? SPA (Single Page Application) concept, component-based architecture, React vs traditional JS.	
56	Vite setup, project structure, JSX rules.	Create first React app: npm create vite@latest
57	Functional components, component structure, JSX rules in depth.	
58	Props, children, reusable components.	Student Card System (start)
59	Component composition patterns, breaking UI into components.	Complete Student Card System
60	State: what is state? useState, updating state.	
61	State vs props, conditional rendering.	
62	Rendering lists, keys.	Counter App
63	Applying state to a real UI.	Product Cart (state-only version, no Redux yet)
64	Event handling in React, controlled components.	
65	Form state, input handling, multiple inputs, submit handling.	
66	Form validation in React.	React Registration Form
67	useEffect: side effects, the dependency array.	
68	useEffect with API calls, loading state, error state.	API Product Dashboard

MODULE 11: Advanced React

Days 69-76 | 16 Hours | 2 Hours/Day

Day	Topics to be Covered	Practical / Output
69	React application architecture: component hierarchy, props drilling, reusable components, folder structure, separation of concerns.	
70	Environment variables (.env), API service files, recommended project structure (src/components, pages, layouts, services, hooks, utils, assets, App.jsx, main.jsx).	
71	React Router: routing concept, BrowserRouter, Routes, Route, Link, NavLink.	
72	useNavigate, useParams, nested routes.	Build routes: /, /products, /products/:id, /about, /contact
73	Advanced Hooks: useRef, useMemo, useCallback — when to use them, not just syntax.	
74	Custom Hooks: creating and using your own hooks.	Build a custom hook, e.g. useFetch
75	API integration in React: Axios setup, GET/POST/PUT/DELETE, API service layer, Component → Service → API → Response → State → UI architecture.	
76	React best practices: reusable components, component composition, avoiding unnecessary state, clean code, error boundaries concept, performance basics, React DevTools, debugging.	Mini Project 3: React Employee Management Dashboard (start)

MODULE 12: Redux Toolkit

Days 77-81 | 10 Hours | 2 Hours/Day

Day	Topics to be Covered	Practical / Output
77	Complete the Employee Management Dashboard. Why state management? Local state vs global state.	
78	Redux architecture: Store, Slice, Reducer, Action, Dispatch, Selector (with diagram).	
79	Redux Toolkit practical setup: configureStore, createSlice, Provider, useSelector, useDispatch.	
80	Async operations in Redux Toolkit; introduction to Redux Toolkit Query (RTK Query).	
81	Applying Redux Toolkit to a real feature.	Shopping Cart: add product, remove product, increase/decrease quantity, total amount, clear cart

MODULE 13: Testing — Jest & React Testing Library (New)

Days 82-84 | 6 Hours | 2 Hours/Day

Day	Topics to be Covered	Practical / Output
82	Why testing matters. Manual UI testing, browser DevTools, console debugging, Network tab.	
83	Introduction to Jest and React Testing Library — writing a simple unit test for a component.	

Day	Topics to be Covered	Practical / Output
84	Applying testing to earlier projects.	Write 2-3 basic tests for existing components (e.g. Counter, Cart)

MODULE 14: Next.js Awareness (New)

Days 85-86 | 4 Hours | 2 Hours/Day

Day	Topics to be Covered	Practical / Output
85	What is Next.js? SSR vs CSR vs SSG concept, file-based routing overview.	
86	Hands-on: create a basic Next.js page and compare its workflow with the React + Vite setup used so far (awareness level only).	

MODULE 15: Capstone Project + Deployment

Days 87-96 | 20 Hours | 2 Hours/Day

Day	Topics to be Covered	Practical / Output
87	Project planning — choose the E-Commerce Frontend capstone, finalize feature list and tech stack (HTML, CSS, JS, React, Router, Redux Toolkit, REST API, Git, GitHub).	
88	Project UI — Navbar, Footer.	
89	Project UI — Home page, Product cards, responsive layout.	
90	Project UI — Product details page.	
91	API integration — fetch products, product details, search.	
92	API integration — filtering, loading state, error handling.	
93	Redux integration — Cart, Wishlist.	
94	Redux integration — User state, Product state. Build Login UI, Register UI, Profile, Orders, 404 page.	
95	Testing & optimization — fix broken UI, console errors, API errors, responsive issues; accessibility checks, performance basics.	
96	Deployment — build process, production build, environment variables, deploy to Vercel/Netlify.	Finalize GitHub repository + README for the capstone project

MODULE 16: Revision, Mock Interview & Presentation

Days 97-100 | 8 Hours | 2 Hours/Day

Day	Topics to be Covered	Practical / Output
97	Full course revision — HTML, CSS, JavaScript quick recap and doubt clearing.	
98	Full course revision — React and Redux Toolkit quick recap and doubt clearing.	
99	Mock technical interview / viva preparation — common frontend interview questions, live coding practice.	
100	Final Project Presentation Day — students present their E-Commerce capstone project and GitHub portfolio; course wrap-up and pointers for further learning (TypeScript deep dive, Next.js, GraphQL, testing frameworks, performance optimization).	Deliverable: GitHub Repository + Live Deployed Website + README + Project Presentation

Final outcome: after 100 days, a student should independently be able to take a requirement such as 'Build a responsive product management frontend that consumes REST APIs, supports authentication UI, has routing, global cart state, validation and deployment' and break it down into UI Design → HTML/JSX → CSS/Responsive UI → JavaScript → React Components → Routing → API Integration → Redux State → Testing/Debugging → GitHub → Deployment.